

Qwen3-VL PAN/ID Extraction — Architecture

Sep 24, 2026 · @infra

Overview

This is an on-device, fully offline pipeline that extracts structured fields (name, ID number, date of birth, etc.) from photographed Indian government ID documents using Qwen3-VL-2B-Instruct, running entirely on the phone's CPU with no cloud calls.

Target hardware: Samsung Galaxy M35 5G and other Exynos 1380 devices — octa-core big.LITTLE (4x Cortex-A78 @2.4GHz "big", 4x Cortex-A55 @2.0GHz "little"), Mali-G68 MP5 GPU (Vulkan 1.2, no cooperative-matrix support).

- This document covers only the Qwen3-VL-2B path. The same Android test app also runs VisionPsy-Nano-460M and PaddleOCR-VL-1.6 on separate code paths, which are out of scope here.

Model & inference stack

Model: Qwen3-VL-2B-Instruct, GGUF Q4_K_M quant (1.1 GB) plus a separate vision-encoder projector (mmproj, Q8_0 quant, 425 MB) — `Qwen3VL-2B-Instruct-Q4_K_M.gguf` / `mmproj-Qwen3VL-2B-Instruct-Q8_0.gguf`.

Engine: a locally patched llama.cpp fork (`qvac-visionpsy-nano/llama-cpp-inference/llama.cpp-custom`) with mtmd multimodal support, built CPU-only for arm64-v8a (armv8.2-a+dotprod+fp16, KleidiAI kernels).

Mode	Binary	Behavior
CLI	<code>libllama-mtmd-cli.so</code>	One process per extraction; always a full cold model load (~90-170s of total latency is just the reload)
Server	<code>libllama-server.so</code>	One persistent process reused over loopback HTTP across requests; stays warm as long as its launch config doesn't change

All the latency-optimization work described below targets server mode.

Document types & extraction pipeline

Doc type	What it is	Status
<code>pan</code>	Baseline full-card PAN prompt, full system prompt, 1024-token cap	Superseded by <code>pan_full</code> for latency work, still present
<code>pan_full</code>	Optimized full-card PAN extraction (all 5 fields in one shot): lean prompt, no system prompt, 128-token cap	This cycle's main focus
<code>pan_crop</code>	Single-photo crop of any one of PAN number/name/DOB/parent name, field auto-detected by shape, no grammar, 256-token cap	Validated
<code>aadhaar</code> , <code>passport</code> , <code>d1</code>	Full-card extraction, original three doc types	Still on unoptimized 1024-token default
<code>tt_passport</code> , <code>tt_d1</code> , <code>tt_id</code>	Trinidad & Tobago passport / driving permit / national ID (<code>tt_id</code> splits across front+back photos)	Added 2026-09-16, not yet hardened
<code>pan_stage1</code> , <code>aadhaar_stage1</code>	Evaluates a LoRA fine-tune ("stage1-contract"); grammar off so the model's own raw parse-failure rate can be measured	Evaluation only

Pipeline for one extraction:

1. User picks doc type and image(s) in the app.
2. App loads `assets/prompts/user_<docType>.txt` and `assets/schemas/<docType>.json` for that type.
3. Unless the doc type skips it, `assets/prompts/system.txt` is prepended as a system message (JSON-only output, English/Latin script only, never invent characters, null for unreadable or missing fields, no transliteration).
4. Request goes to the running (or freshly launched) inference process with the image, prompt, and schema.

- 5. The schema is compiled to a GBNF grammar (`-jf` flag), constraining decoding to valid, schema-conforming JSON only.
- 6. Output is parsed, run through a doc-type-specific validator (structural regex/checksum/cross-field checks), and a leak-detector that flags any value matching a known prompt-example or template-label leak.
- 7. Result and elapsed time are shown in the app.

The image-token cap — the core latency/accuracy lever

Set via the env var `LLAMA_ARG_IMAGE_MAX_TOKENS` , read once by the vision encoder at process start (`clip.cpp` 's `set_limit_image_tokens()` , Qwen-VL family only, valid range 8-4096). It bounds how many vision tokens an image is resampled into before the language model sees it: fewer tokens means less prefill compute (lower latency) but less surviving fine detail (risk to small print like PAN digits or a date of birth).

Because it's read once at startup and not per request, it can't change on an already-running server — a different cap means killing and relaunching the process (see the caching section below for why that matters).

Doc type(s)	Cap	Basis
<code>pan_full</code>	128	On-device sweep from 128 to 1024 on one test card: 5/5 fields exact at every level; 128 chosen for lowest latency. Real-card testing since found misreads on 2 of 3 fresh cards at this cap — open regression, see Current status.
<code>pan_crop</code>	256	Validated cold (21.2s) and warm (0.87s) on a real crop, exact match; 1024 measurably worse (4/6 vs 6/6 correct)
<code>pan</code> , <code>aadhaar</code> , <code>passport</code> , <code>dl</code> , <code>tt_*</code> , <code>stage1</code> types	1024	Carried-over default, not yet given the same on-device validation

The applyDynSizeEnv bug (found and fixed this cycle)

Qwen3-VL and PaddleOCR-VL share the same image-preprocessing function in this fork, `mtmd_image_preprocessor_dyn_size` (`mtmd-image.cpp`). It computes min/max-pixel bounds from the token cap, classifies the image as "full card" or "tight crop", then — only if two specific env vars are set — overwrites (not clamps) those bounds.

Those two env vars (`MTMD_CROP_THRESHOLD_PIXELS`, `MTMD_FULLIMAGE_MAX_PIXELS`) were meant for PaddleOCR-VL only, but the function that sets them (`applyDynSizeEnv`) was being called unconditionally for every model. Result: Qwen's own token-cap setting was silently discarded for every "full image" (non-crop) request.

Confirmed on-device (2026-09-24): `pan_full` produced byte-identical prompt token counts (1398) at cap settings of 1024, 512, and 256 — proof the cap was doing nothing — until the call was gated to PaddleOCR-VL only.

Fix: both call sites (`runOnePass`, `ensureServerRunning`) now guard the call with `if (isPaddle0cr)`.

Other performance tuning validated on-device

- CPU core pinning: `-Cr 4-7 -Crb 4-7 --cpu-strict 1 --cpu-strict-batch 1` pins inference threads to the 4 "big" Cortex-A78 cores (indices 4-7, confirmed via `/proc/cpuinfo` CPU part IDs). Without it, the scheduler let threads migrate onto the slower A55 cores, producing 8-16 tok/s prefill-rate swings between otherwise-identical requests.
- Flash-attention: `-fa on` forced. The default `auto` was landing off in practice; forcing it on measurably improved prefill throughput.
- GPU offload: off, by design. Vulkan GPU offload was built and tested (`-ngl 99 --no-mmproj-offload`) — it ran slower than CPU-only, because the Mali-G68 MP5 lacks the `GL_KHR_cooperative_matrix` Vulkan extension llama.cpp's fast matmul path needs, falling back to a slow generic compute-shader path. This was measured on the sibling VisionPsy-Nano model, but it's a hardware/driver limitation, not a model-specific one, so it applies to Qwen too — the toggle stays off.

Server-mode caching, precisely — two different mechanisms

These get confused for each other; they're unrelated.

1. Prompt cache (`cache_prompt` , on by default in llama-server): reuses the previous request's prefix tokens if they match exactly. Bounded by context size, overwritten each request, never grows. This is why a second identical request is fast ("warm") and a genuinely new document is slower ("cold") — nothing to fix here, it already works as intended.
2. Process-reload-on-config-change (`ensureServerRunning`): the server process is reused only if model, mmpoj, threads, GPU, and image-token cap all match the previous request. Since image-token cap differs by doc type (128 / 256 / 1024), switching doc types always fails this match and forces a full process kill plus cold reload, even between two back-to-back requests. This is the real cause of "why did it go cold again", not memory buildup: at most one server process ever exists, since the old one is destroyed before the new one starts.

No fix implemented yet. Candidate options, cheapest to most invasive:

- Unify the cap across doc types — loses per-type tuning.
- Run one persistent server per distinct cap concurrently — costs about 1.5 GB RAM per extra copy (Qwen3-VL-2B plus mmpoj alone is 1.5 GB).
- Patch the fork to make the cap a per-request parameter instead of process-level — unscoped, needs `clip.cpp` / `mtmd.cpp` investigation.
- Accept the reload cost as-is.

Current status: measured latency & accuracy

`pan_full` latency:

Stage	Cold	Warm
Original (1024-token cap, full system prompt, no CPU pinning)	~127-180s	~50-90s
Current (128-token cap, lean prompt, CPU pinning, forced flash-attn)	~33-45s	~10-30s

`pan_full` accuracy: on the one synthetic test card used for the 128-2046 token sweep, every level through 1024 was 5/5 fields exact. On 3 freshly tested real ground-truthed cards at the current 128-token cap, 2 of 3 had wrong names or parent-names, and one had a wrong date of birth — an open, unreconciled regression between the sweep result and real-card behavior.

`pan_crop` : 256-token cap, validated cold (21.2s) and warm (0.87s), exact match on a real crop.

Open items and decisions not yet made

- Reconcile the 128-token `pan_full` accuracy regression seen on real cards against the clean sweep result.
- Decide and implement a fix for the doc-type-switch reload problem (options above).
- Run the prepared blur/degradation robustness test (5 real cards, human-readable Gaussian blur, ground truth already on hand) — blocked on the token cap not being exposed as a UI control.
- Complete the two sweep levels never run (1536, 2046) and repeat the sweep methodology on more than one image.
- Explore batch/ubatch size and `--mtmd-batch-max-tokens` tuning — not started.
- `aadhaar`, `passport`, `dl`, `tt_passport`, `tt_dl`, `tt_id`, and the stage1 doc types are all still on the pre-optimization 1024-token default and haven't had the same on-device validation `pan_full` and `pan_crop` got.